大数据应用人才培养系列教材

Python语言

刘 鹏　张 燕　○总主编
李肖俊 ○主编　　刘 河　钟 涛 ○副主编

清華大學出版社
TSINGHUA UNIVERSITY PRESS

第九章　异常

9.1 异常概述

9.2 异常处理流程

9.3 自定义异常

9.4 实验

9.5 小结

9.6 习题

9.1.1 认识异常

异常就是一个事件，该事件会在程序执行过程中有语法等错误的时候发生，异常会影响程序的正常执行。

通常在Python无法正常处理程序时就会发生一个异常，程序会终止执行。

当Python程序发生异常时，我们需要检测捕获处理它。

9.1.2 处理异常

如何检测并处理异常呢?

我们检测处理异常，可以通过try语句来实现，任何出现在try语句范围内的异常都可以被检测到，有4种模式的try语句： try —except语句、try—except—finaly语句、try—except—else语句、try (with)—except语句。

(1) Try — except语句:

语法格式如下:

```
try:
    [语句块]
except Exception[as reason]:
```

9.1.2 处理异常

出现异常(exception)后的处理代码

在语法格式中，“[语句块]”属于try语句的检测范围，它类似于while循环、for循环、if/else语句一样；“except”后面跟上一个异常的名字，“as reason”报出异常的具体内容，并把这详细异常信息输出；“出现异常(exception)后的处理代码”这部分是程序员对出现异常后如何处理的代码。

一个try语句还可以和多个except语句搭配,对我们感兴趣的异常进行检测处理。

如果try语句包含的异常没有出现在后面跟着的except语句中的时候，则程序直接报错输出异常的类型。

9.1.2 处理异常

如果当我们不确定在try语句块中会出现哪一种异常的时候，我们可以在except后面不跟具体的异常类型。

上面这种处理方式不推荐采用，因为这样做会掩藏程序员未想到的所有未曾处理过的错误。

有一点一定要注意：try语句检测范围一旦出现了异常，剩下的其它语句将不会被执行。如上例中，程序运行到try语句块的第一条语句“int('det')”的时候检测到异常，程序立即跳转到except执行异常处理程序，其它程序代码块就不再执行了。

另外，如果要对多个异常进行统一的处理，采用如下的语法格式：

9.1.2 处理异常

```
try:
    [语句块]
except (Exception1，Exception2，Exception3，…):
    出现异常(exception)后的处理代码
```

在上述语法中，多个异常之间用逗号“，”隔开。

9.1.2 处理异常

(2) try - finally语句：

语法格式如下：

```
try:
    [语句块]
except Exception[as reason]:
    出现异常(exception)后的处理代码
finally:
    无论如何都会被执行的代码
```

9.1.2 处理异常

在上述语法中，如果一旦检测到try语句块中有任何异常，程序就会根据异常类型跳转到except处执行对应异常类型的处理代码，最后再跳转到finally处执行里面的代码；如果在try语句块中没有检测到任何异常，程序在执行完try语句块里的代码后，跳过except中的语句块，最后跳转到finally处执行里面的代码。

(3) try – except - else语句：

语法格式如下：

try:

    [语句块]

9.1.2 处理异常

except Exception[as reason]:

出现异常(exception)后的处理代码

else:

没有异常后被执行的代码

在上述语法中，如果一旦检测到try语句块中有任何异常，程序就会根据异常类型跳转到except处执行对应异常类型的处理代码，最后终止程序的执行；如果在try语句块中没有检测到任何异常，程序在执行完try语句块里的代码后，跳转到else处执行里面的代码。

9.1.2 处理异常

(4) try (with)—except语句：

语法格式如下：

```
try:
    with <语句> as name:
 [语句块]
except Exception[as reason]:
    出现异常(exception)后的处理代码
```

在语法中可见，with语句出现在try语句块中，一般情况下就不用再写finally语句块了。使用with语句的最大好处是减少代码量，比如当我们对文件操作时忘记了关闭文件操作，则with语句会自动执行关闭文件操作。

第九章　异常

9.1 异常概述

9.2 异常处理流程

9.3 自定义异常

9.4 实验

9.5 小结

9.6 习题

9.2 异常处理流程

Python中的异常处理流程是：

当程序运行try语句块检测到异常时，立即终止有异常的语句的执行，跳转到匹配该异常的except子句执行异常处理代码，异常处理完毕后，如果有finally语句就执行该语句块中的代码，最后终止整个程序的执行，如果没有finally语句就直接终止整个程序的执行。

如果检测到异常，但没有该异常匹配的except子句，分两种情形：如果有finally语句就执行该语句块中的代码，最后终止整个程序的执行；如果没有finally语句就直接终止整个程序的执行。

如果在try语句块中没有检测到异常，程序执行完try语句块后，如果有else语句块就执行里面的内容最后控制流就通过整个try语句，没有else语句控制流就直接通过整个try语句。

第九章　异常

9.1 异常概述

9.2 异常处理流程

9.3 自定义异常

9.4 实验

9.5 小结

9.6 习题

9.3 自定义异常

自定义一个简单的异常类，如下所示：

```
class MyError(Exception):
    pass
```

在程序代码中使用关键字raise来抛出自定义的异常。

语法格式如下：

```
raise MyException(defineexceptname)
```

在上述语法中，MyException——自定义异常的类型，defineexceptname——自定义异常的说明。

同时，我们也可以结合try—except主动抛出自定义的异常，如下例代码所示：

```
class Myerror(Exception):
        pass
try:
    raise Myerror('测试自定义的异常')
except Myerror as e:
    print(e)
```

运行结果如下：

```
测试自定义的异常
```

第九章　异常

9.1 异常概述

9.2 异常处理流程

9.3 自定义异常

9.4 实验

9.5 小结

9.6 习题

9.4 实验

9.4.1 利用try-except处理除数为零的异常

9.4.2 自定义异常的使用

9.4.3 raise关键字的使用

9.4.4 内置异常处理语句的使用

第九章　异常

9.1 异常概述

9.2 异常处理流程

9.3 自定义异常

9.4 实验

9.5 小结

9.6 习题

9.5 小结

在Python当中，若程序在运行时出错，系统会自动地在出错的地方生成一个异常对象，而后系统会在出错的地方向后寻找是否有对这个异常对象处理的代码，如果没有，系统会将这个异常对象抛给其调用函数，这样层层抛出，如果在程序主函数中仍然没有对这个异常对象处理的代码，系统会将整个程序终止，并将错误的信息输出。

第九章　异常

9.1 异常概述

9.2 异常处理流程

9.3 自定义异常

9.4 实验

9.5 小结

9.6 习题

习题：

1. Python异常处理结构有哪几种形式？
2. 异常和错误是同一概念吗？为什么？

感谢聆听